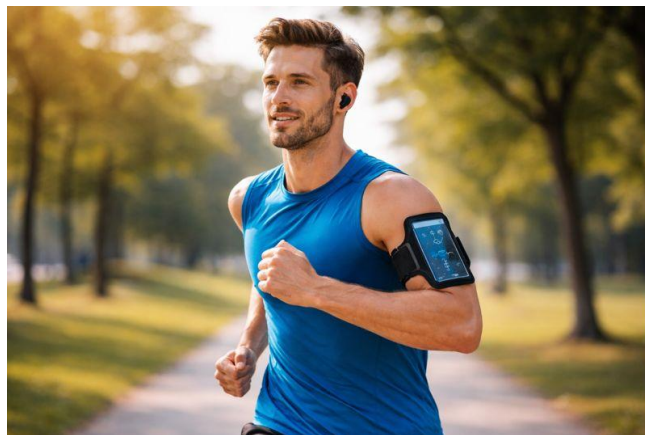




הצעת פרויקט לעבודת גמר י"ד הנדסאי תוכנה (שאלון 714918)

בנושא

אפליקציית מעקב ריצה



שם הסטודנט: שליו אבקסיס

תעודת זהות: 330589664

שם המנחה: אילן פרץ

תאריך: 13/1/2026

מכללה: כנפי רוח, קריית נוער ירושלים (סמל מוסד: 140129)

תוכן עניינים:

1.	תיאור הנושא.....	3
2.	רקע תיאורטי בתחום הפרויקט.....	4
3.	תיאור הפרויקט.....	7
4.	הגדרת הבעיה האלגוריתמית.....	10
5.	הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות.....	13
6.	הליכים עיקריים בתחום למידת מכונה - לא רלוונטי.....	16
7.	הליכים עיקריים במע' הפעלה/רשתות/תקשורת/אבטחה - לא רלוונטי.....	16
8.	תיאור פרוטוקולי תקשורת.....	16
9.	פיתוחים עתידיים.....	16
10.	תיאור טכנולוגיית הנדסה.....	17
11.	מסד נתונים.....	18
12.	פרטים פורמליים.....	20

1. תיאור הנושא

הפרויקט עוסק בתחום מערכות למעקב וניתוח ריצה מבוסס מיקום. מטרת התחום היא לתעד ריצה בזמן אמת, לנתח ולהציג מידע עבור מאפייני הריצה והמסלול.

מעקב הריצה מתבצע בדרך כלל באמצעות GPS. תוך כדי הריצה, המערכות אוספות נתונים באופן רציף לאורך המסלול, כמו: קצב נוכחי, מרחק שעבר עד אותה נקודה, זמן נוכחי וכדומה. בסיום המעקב מחושבים נתונים סטטיסטיים שנאספו במהלך הריצה עצמה ועל ידי סטטיסטיקות אלו ניתן לקבל אינדיקציה ומסקנות בנוגע לביצוע ולאיכות הריצה.

תחום מעקב וניתוח ריצה על בסיס מיקום מביא עימו מספר אתגרים. האתגר ראשון הוא נתוני GPS אשר עלולים להיות מושפעים מתנאי סביבה כגון מבנים גבוהים, אזורים סגורים או קליטה חלשה, דבר העלול לגרום לסטיות במסלול, בשקלול המרחק או בקצב הריצה.

אתגר נוסף הוא עיבוד וניתוח הנתונים בזמן אמת - האפליקציות נדרשות לחשב מרחק, מהירות, קצב וסטטיסטיקות נוספות תוך כדי תנועה, מבלי לפגוע בביצועי המכשיר או בצריכת הסוללה.

מעבר לכך, הפרויקט עוסק בתחום מציאת מסלול מיטבי תחת אילוצים (ונקרא Orienteering). מטרת תחום זה היא לתכנן ולמצוא מסלול מיטבי בין נקודת התחלה לנקודת סיום, תוך בחירה מושכלת של נקודות ביניים על סמך ערך של כל נקודה המעיד על איכותה, כך שהמסלול יעמוד במגבלות ואילוצים נתונים. ויביא להשגת הערך המצטבר המרבי ביותר שניתן להשיג.

בעקבות כך, תחום זה כולל בתוכו מספר מורכבויות ואתגרים. אחד האתגרים המרכזיים בתחום הוא מורכבות חישובית גבוהה - ככל שמספר הנקודות האפשריות גדל, מספר המסלולים האפשריים גדל בצורה משמעותית, מה שהופך את מציאת הפתרון האופטימלי למשימה מורכבת מאוד מבחינה חישובית.

אתגר נוסף הוא שילוב של מספר אילוצים. מציאת המסלול המיטבי דורשת התחשבות במספר גורמים בזמנית, כגון מגבלת זמן, מהירות, מרחק ועוד. שילוב אילוצים מקשה על מציאת פתרון פשוט וברור.

בעיה מרכזית נוספת היא היעדר פתרון מדויק בזמן סביר. במקרים רבים, במיוחד כאשר מספר הנקודות גדול, לא ניתן לחשב פתרון אופטימלי בזמן קצר.

2. רקע תיאורטי בתחום הפרויקט

תחום המערכות למעקב וניתוח ריצה על בסיס מיקום הינו מעטפת לתחום העיקרי של הפרויקט שהוא מציאת מסלול מיטבי תחת אילוצים. לתחום מציאת המסלול המיטבי נחשפתי לאחר שחיפשתי ברחבי האינטרנט אלגוריתם אשר מתייחס לבעיה ופותר אותה. במהלך המחקר נחשפתי לעולם של בעיות המסלול ובפרט לבעיית המסלול שבה הפרויקט עוסק - Orienteering. מקור השם Orienteering הוא בענף ספורט המתקיים בדרך כלל באזורים מיוערים. ביער ממוקמות מספר "נקודות בקרה" שלכל אחת מהן ציון משויך. למתחרים יש מצפן ומפה והם נדרשים לבקר בתת-קבוצה של נקודות בקרה מנקודת ההתחלה כדי למקסם את הציון הכולל שלהם ולחזור לנקודת הסיום בתוך פרק זמן שנקבע מראש כך שמתחרים שמגיעים באיחור נפסלים. כיום יש מספר מערכות מוכרות המאופיינות עם בעיית ה-Orienteeing, כמו Waze, Wolt, Wanderlog וכדומה.

מעבר לבעיה שבה הפרויקט עוסק, ישנן מספר בעיות נוספות היושבות תחת קטגוריית בעיות המסלול:

בעיית הסוכן הנוסע (Traveling Salesman Problem) - מתייחסת לבעיה בה צריך למצוא מסלול מינימלי שעובר בדיוק פעם אחת בכל נקודה, וחוזר לנקודת ההתחלה. לבעיה זו יש תת גרסאות עבור מקרים ספציפיים כגון: Prize-Collecting TSP - בעיית TSP עם איסוף פרסים (ניקודים) ועמידה במכסת סכום הפרסים שניתן לצבור תוך שמירה על אורך מסלול קצר ביותר, k -TSP - יש k סוכנים שצריכים לבקר בכל הצמתים מתחנת מוצא משותפת, והמטרה היא לחלק ביניהם את הצמתים כך שסך אורכי המסלולים (או המסלול הארוך ביותר) יהיה מינימלי.

בעיית ניתוב רכבים (Vehicle Routing Problem) - מתייחסת לבעיה בה צריך לתכנן מסלולים עבור כמה שליחים ולא עבור אחד, כך שכל היעדים יקבלו שירות תוך עמידה במגבלות (כמו: קיבולת, שעות, מרחק, אנרגיה). גם לבעיה זו יש מספר וריאציות כמו: Capacitated VRP - בעיית VRP שבה לכל רכב יש קיבולת מוגבלת (למשל נפח משאית או מספר חבילות) ויש לספק את כל הלקוחות או לנקודות היעד, תוך שמירה על מגבלת הקיבולת, ולמזער עלות/מרחק כולל, Multi-Depot Vehicle Routing Problem - בעיית VRP שבה יש מספר נקודות התחלה ויש לתכנן מסלולים עבור כל רכב שיוצא מנקודת התחלה מסוימת, וכו'.

בעיית המסלול הקצר ביותר (Shortest Path Problem) - מתייחסת למציאת המסלול הקצר ביותר עבור גרף נתון. יש מספר אלגוריתמים העוסקים בפתרון בעיה זו (עם מעט הבדלים) כמו: Dijkstra - הוא אלגוריתם למציאת המסלול הקצר ביותר מצומת התחלה אחת אל כל שאר הצמתים בגרף שבו המשקלים אינם שליליים. Bellman-Ford - הוא אלגוריתם למציאת המסלול הקצר ביותר

מצומת מקור יחיד לכל שאר הצמתים בגרף עם משקולות קשתות (כולל משקולות שליליות). Floyd-Warshall הוא אלגוריתם למציאת המסלולים הקצרים ביותר בין כל זוג צמתים בגרף.

גם בתוך בעיית ה-Orienteering יש מספר וריאציות: Orienteering with Variable Profits - זר וריאציה, שבה הרווח (profit) של כל נקודה אינו קבוע וידוע מראש, אלא משתנה בהתאם לנסיבות מסוימות, במהלך המסלול Team Orienteering - וריאציה שבה יש כמה שליחים, ולכל אחד צריך למצוא מסלול עם מגבלת זמן/מרחק, כך שסך הרווח מכל המסלולים ביחד יהיה מקסימלי.

התחום בו המערכת שלי מתרכזת הוא תחום ה-Orienteering הקלאסית כפי שהגדרנו בהגדרת הבעיה האלגוריתמית (ונקראת גם Generalized TSP). מאחר והבעיה אינה ניתנת לפתירה בזמן סביר, ישנם מספר אלגוריתמי שיפור שהוצעו במהלך היסטורית מחקר הבעיה:

אלגוריתם גנטי (Genetic Algorithm) - אלגוריתם גנטי הוא אלגוריתם בהשראת תהליכים אבולוציוניים בטבע. הוא פועל על אוכלוסייה של פתרונות אפשריים, כאשר כל פתרון מכונה פרט (Individual) ומיוצג בדרך כלל כמחרוזת גנים (למשל וקטור, רצף צמתים, ביטים וכו'). בכל איטרציה האלגוריתם מבצע "אבולוציה": הוא מעריך את איכות כל פרט לפי פונקציית מטרה, בוחר את הפרטים הטובים ביותר ("שורדים"), יוצר צאצאים באמצעות Crossovers (שילוב בין גנים של שני פרטים איכותיים), ומבצע Mutations (שינויים אקראיים קלים) כדי למנוע קיבעון ולאפשר חיפוש גלובלי. כך נוצרות אוכלוסיות חדשות שהן לרוב טובות יותר מהקודמות. אלגוריתם גנטי מתאים לבעיות גדולות ומורכבות, מסוגל לברוח ממינימום מקומי, עובד היטב על בעיות קומבינטוריות כמו מסלולים וגרפים, ומאפשר שילוב קל עם טכניקות Local Search. **חסרונות:** אינו מבטיח אופטימליות, רגיש להגדרות פרמטרים (גודל אוכלוסייה, שיעור מוטציה וכו'), עלול להיות איטי אם לא מכוונן היטב, ולעיתים זקוק להרצות רבות כדי להבטיח פתרון איכותי.

Branch & Cut - הוא אלגוריתם מדויק (Exact), כלומר שואף למצוא את הפתרון האופטימלי המוחלט לבעיה קומבינטורית, ואחד מהכלים המרכזיים לפתרון בעיות NP-Hard כמו TSP, Orienteering, VRP ועוד. הוא מבוסס על פתרון אינטגרלי של בעיות תכנות ליניאריות בשלמים (ILP). תחילה האלגוריתם פותר את גרסת ה-LP relaxation - כלומר מתיר למשתנים לקבל ערכים רציפים בין 0 ל-1. התוצאה היא חסם עליון לפתרון הטוב ביותר. לאחר מכן בודקים אם הפתרון רציף (לא שלם). אם כן - האלגוריתם מבצע Branching, מפצל את הבעיה לשתי תתי-בעיות ומוסיף אילוצים שמכריחים את המשתנים להתקבע לערכים אינטגרליים (0 או 1). במקביל האלגוריתם מבצע Cutting Planes - מוסיף אילוצי חיתוך שמטרתם לחתוך אזורים לא חוקיים במרחב ה-LP ולשפר את החסם. התהליך נמשך עד שכל העלים בעץ נותנים פתרונות שלמים או נחתכים. אלגוריתם זה מספק פתרון אופטימלי ודטרמיניסטי, נותן הוכחה לרמת האיכות של התוצאה, יעיל

יחסית בזכות Cuts שמקטינים את מרחב החיפוש. עם זאת, זמן הריצה שלו אקספוננציאלי במקרה הגרוע, הופך לאיטי מאוד בבעיות עם מאות צמתים, דורש הרבה זיכרון, ומרגיש "כבד" לתפעול בעולם הפרקטי כשצריך תוצאה מהירה.

אלגוריתם חמדני (Greedy Algorithm) - אלגוריתם חמדני הוא אלגוריתם שבכל צעד בוחר את האפשרות שנראית הטובה ביותר *ברגע הנוכחי*, בלי להתחשב בעתיד או בתמונה הכוללת. ברוב המקרים הוא פועל בצורה מאוד פשוטה: מתחילים מפתרון ריק, ובכל שלב מוסיפים את הרכיב הבא עם הערך הטוב ביותר (למשל הצומת בעל הרווח הגבוה ביותר ליחידת זמן, הקשת הקצרה ביותר, או הבחירה עם העלות המינימלית). ההחלטות הן לוקליות בלבד, ללא תיקון טעויות ובלי חזרה אחורה. האלגוריתם מהיר ביותר (כמעט תמיד זמן פולינומי פשוט), קל ליישום, אינו דורש פרמטרים או כיוונון, ופועל טוב כבסיס לפתרונות התחלתיים באלגוריתמים מתקדמים (כמו PSO, GA, B&C). אך עם זאת, הוא עלול להיתקע בפתרון גרוע, אינו מבטיח אופטימליות, לא יודע "להתחרט", תלוי באופן קריטי בפונקציית הבחירה שלו, ולעיתים קרובות מספק פתרון באיכות הרבה פחות טובה בהשוואה לאלגוריתמים מתוחכמים יותר.

Discrete Particle Swarm Optimization - האלגוריתם DPSO הוא אלגוריתם מטא-היוריסטי אבולוציוני בהשראת התנהגות של להקות ציפורים ודיוני דגים. ה-DPSO הוא גרסה מותאמת של PSO שמיועדת לעבוד על בעיות שבהן הפתרונות הם דיסקרטיים, כמו בבעיית ה-Orienteering שבה כל פתרון אפשרי הוא "חלקיק" (מסלול) שמורכב מרשימת צמתים בתוך מרחב של פתרונות קיימים (אוכלוסייה). פונקציית המטרה (fitness) של כל חלקיק היא סכום הניקוד הכולל של הצמתים שהוא מבקר בהם, כך שהמטרה היא למקסם את סכום הניקוד תוך עמידה במגבלת המסלול T_{max} . הפתרון מיוצג על ידי ה-position של החלקיק שהוא אוסף של הצמתים שבהם המסלול מבקר על פי סדר הביקור. בכל איטרציה האלגוריתם שואף לייצר מסלול טוב יותר על בסיס שילוב בין ניסיון עבר וניסיון קולקטיבי של כל החלקיקים.

לעומת זאת, PSO רגיל מתאים רק למרחבים רציפים שבהם אפשר להגדיר מהירות, כיוון ותנועה וקטורית, אך בעיית המסלול אינה מאפשרת לבצע פעולות כאלה, אי אפשר "להזיז" מסלול בצורה מתמטית כמו שמזיזים נקודה במרחב. לכן DPSO משתמש במקום במנגנון הסתברותי שמרכיב מסלולים חדשים מתוך שילוב של המסלול הנוכחי, המסלול הטוב שהחלקיק מצא בעבר, והמסלול הטוב ביותר שנמצא בכלל האוכלוסייה. באופן הזה האלגוריתם שומר על עקרונות הלמידה של PSO, אבל מתאים אותם למבנה הדיסקרטי של הבעיה. הבחירה ב-DPSO מאפשרת לנצל את היתרונות של PSO - פשטות, ביצועים טובים והתאמה לחיפוש במרחב גדול, תוך התאמה מלאה לבעיה שבה הפתרונות הם מסלולים ולא מספרים רציפים. אלגוריתם ה-DPSO מאפשר לתת איכות פתרון טובה

ביחס לזמן הריצה שלו, ללא מימוש מורכב. אך למרות זאת, האלגוריתם לא מספק בהכרח את הפתרון האופטימלי, רגיש לכוונון פרמטרים ועלול להיכנס לעיתים לפתרון מוקדם.

להלן טבלת השוואה וסיכום בין סוגי האלגוריתמים:

אלגוריתם	יתרונות	חסרונות
אלגוריתם גנטי (Genetic Algorithm)	מתאים לבעיות גדולות ומורכבות, ועובד היטב על בעיות קומבינטוריות (מסלולים, גרפים).	אינו מבטיח אופטימליות, רגיש לכוונון פרמטרים (גודל אוכלוסייה, מוטציה), עלול להיות איטי, דורש מספר רב של הרצות לקבלת פתרון איכותי.
Branch & Cut	מספק פתרון אופטימלי ודטרמיניסטי, נותן הוכחה לאיכות הפתרון.	זמן ריצה אקספוננציאלי במקרה הגרוע, לא סקיילבילי לבעיות גדולות, צורך זיכרון גבוה, כבד ולא מתאים למצבים הדורשים פתרון מהיר.
אלגוריתם חמדני (Greedy Algorithm)	מהיר מאוד, פשוט ליישום, אינו דורש פרמטרים או כיוונון.	אינו מבטיח אופטימליות, עלול להיתקע בפתרון גרוע, מבצע החלטות ללא חזרה אחורה, תלוי מאוד בפונקציית הבחירה.
Discrete Particle Swarm Optimization (DPSO)	מספק איזון מיטבי בין איכות הפתרון, זמן הריצה והגמישות, המתאימים במיוחד לאופי הבעיה, וקל יחסית למימוש ולשילוב עם Local Search.	אינו מבטיח פתרון אופטימלי, רגיש לפרמטרים (גודל להקה, מקדמים וכדומה), עלול להתכנס לפתרון בשלב מוקדם.

3. תיאור הפרויקט

הפרויקט יבנה כאפליקציית Android. האפליקציה מיועדת לרצים - חובבים ומתקדמים - המעוניינים לעקוב בצורה נוחה וברורה אחר הריצות שלהם ולבחון את התקדמותם לאורך זמן. האפליקציה מאפשרת למשתמש לתעד ריצות באמצעות GPS, כך שבסיומה של כל ריצה מוצג מסלול הריצה שבוצע בפועל, יחד עם נתונים וסטטיסטיקות המסכמות את הביצועים.

המערכת מנתחת את נתוני הריצות ומציגה אותם בצורה ויזואלית וברורה באמצעות גרפים, טבלאות ואלמנטים גרפיים נוספים, במטרה לאפשר למשתמש להבין את מצבו: האם חל שיפור בביצועים, באיזה היבטים הוא התקדם, ועד כמה הוא קרוב ליעדים שהציב לעצמו. כך יכול המשתמש להסיק מסקנות מעשיות לגבי האימונים שלו ולזהות נקודות לשיפור.

בנוסף למעקב אחר ריצות שבוצעו, האפליקציה מאפשרת למשתמש להזין העדפות למסלול ריצה רצוי, כגון מרחק, הפרש גבהים מקסימלי וקרבה למיקום מסוים. על סמך העדפות אלו, המערכת מציעה למשתמש מסלולי ריצה מתאימים, המדורגים לפי איכות ההתאמה לדרישותיו, ובכך מסייעת לו לבחור מסלול ריצה המתאים ליכולותיו ולמטרותיו.

מעבר לכך, כפי שצוין בסעיף הראשון, ישנם אתגרים נלווים בתחומי הפרויקט ויש להתייחס לאופן פתירתם:

לצורך התמודדות עם בעיית דיוק נתוני המיקום, האפליקציה עושה שימוש בנתוני GPS רציפים ומבצעת סינון ועיבוד של הנתונים על מנת לצמצם סטיות ולשפר את אמינות חישוב המרחק והמסלול.

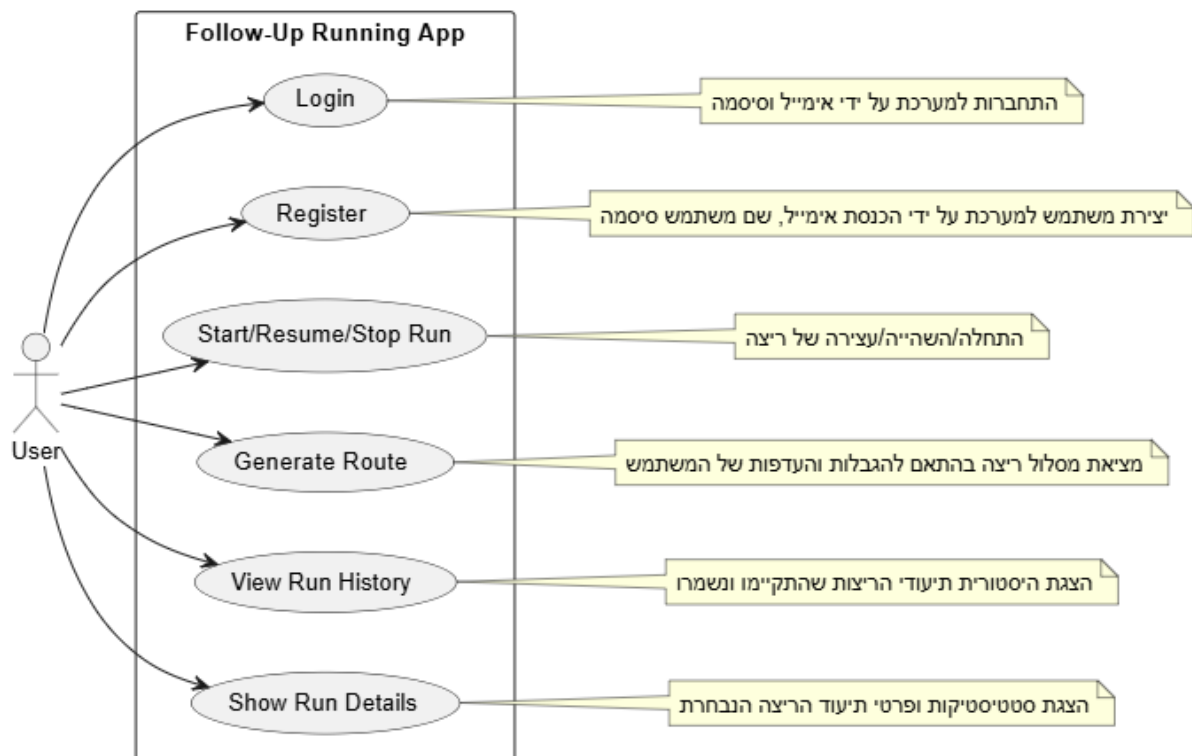
במטרה להתמודד עם האתגר של עיבוד נתונים בזמן אמת תוך שמירה על חיי הסוללה, האפליקציה מבצעת חישובים בסיסיים במהלך הריצה ואוגרת נתונים לניתוח מעמיק יותר לאחר סיום הפעילות, ובכך מאזנת בין דיוק לביצועים.

כדי להתמודד עם בעיית המורכבות החישובית הגבוהה, המערכת אינה שואפת למצוא תמיד את המסלול האופטימלי המוחלט, אלא מסלול איכותי ויעיל שניתן לחשב בזמן סביר. גישה זו מאפשרת שימוש בבעיה גם בסביבה יישומית כמו אפליקציית מובייל.

לצורך התמודדות עם שילוב של מספר אילוצים במציאת המסלול המותאם, במקום להתייחס לכל אילוץ בנפרד, המערכת מתייחסת אליהם כאל סט אילוצים משולב, ומחפשת מסלול העומד בצורה מיטבית באילוצים שהוגדרו, גם אם לא ניתן לממש את כולם באופן מושלם. כך ניתן למצוא פתרון מעשי ואיכותי המשקף פשרה סבירה בין הדרישות השונות.

בשביל להתמודד עם היעדר פתרון מדויק בזמן סביר עבור מציאת המסלול האופטימלי, האפליקציה תעזר באלגוריתם המוצע לפתרון הבעיה, ובמהלך מימוש האלגוריתם יתבצעו שיפורים ליעל אותו.

באפליקציית מעקב הריצה, המשתמש יוכל לבצע מספר פעולות (תהליכים) עיקריים. להלן **תרשים תרחיש (Use Case Diagram)** התחלתי המציג את הפעולות שיוכל המשתמש לבצע באפליקציה (ייתכן ויתווספו עוד פעולות ותהליכים בהמשך):



בתרשים ניתן לראות שיש במערכת 6 תרחשים לעת עתה, להלן הסבר על כל אחד מהם:

- **התחברות (Login)** - מאפשרת למשתמש קיים להתחבר לאפליקציה באמצעות אימייל וסיסמה. לאחר ההתחברות נטענים הנתונים האישיים, היסטוריית הריצות וההגדרות השמורות של המשתמש.
- **הרשמה (Register)** - מאפשרת יצירת משתמש חדש במערכת באמצעות הזנת פרטים כמו אימייל, שם משתמש וסיסמה. בסיום ההרשמה נוצר חשבון אישי לשמירת נתוני הריצות והסטטיסטיקות.
- **התחלה/השהייה/עצירה של ריצה (Start/Resume/Stop Run)** - מאפשרת שליטה מלאה במהלך הריצה: התחלת מעקב, השהייה זמנית והמשך, או סיום הריצה. במהלך הריצה נאספים נתונים כמו זמן, מרחק ומהירות.
- **יצירת מסלול (Generate Route)** - האפליקציה מייצרת מסלול ריצה מומלץ בהתאם להעדפות המשתמש. המסלול מוצג על גבי מפה לפני תחילת הריצה.

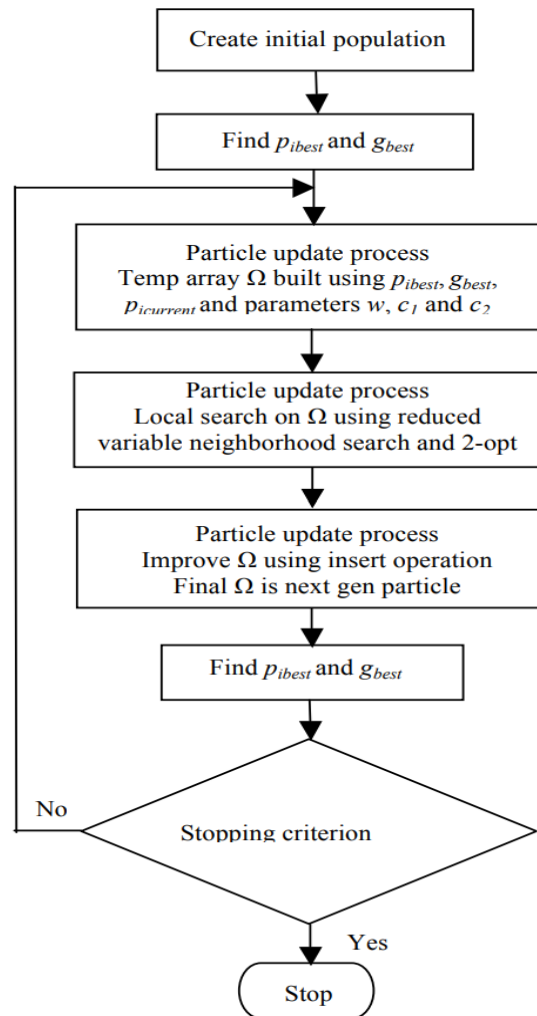
- **הצגת היסטוריית ריצות (View Run History)** - מאפשרת למשתמש לצפות ברשימת כל הריצות שביצע בעבר. ההיסטוריה כוללת תאריכים, מרחקים וזמני ריצה לצורך מעקב אחר התקדמות.
- **הצגת פרטי ריצה (Show Run Details)** - מציגה מידע מפורט על ריצה מסוימת, כגון מסלול על מפה, קצב, מהירות ממוצעת וסטטיסטיקות נוספות. מאפשרת ניתוח ביצועים והשוואה לריצות קודמות.

4. הגדרת הבעיה האלגוריתמית

כפי שצוין, המערכת תיצור עבור המשתמש מסלול אופטימלי למשתמש לפי המגבלות שציב ולפי איכות המסלול. בעיה זו של מציאת מסלול איכותי תחת מגבלות משתייכת לקטגוריית בעיות המסלול (routing problems). הבעיה האלגוריתמית אשר מתייחסת למקרה זה היא בעיית ה-Orienteeing שהיא בעיה בלתי ניתנת לפתירה בזמן סביר (NP-Hard Problem) במשפחת בעיות המסלול שבה צריך למצוא מסלול מיטבי שמתחיל בנקודה ומסתיים בנקודה (בין אם נקודות שונות ובין אם אותה נקודה), כאשר לאורך המרחב קיימים מספר יעדים (נקודות) אפשריים, ולכל יעד יש ניקוד. המטרה היא לבחור תת-קבוצה של יעדים ולתכנן מסלול שעובר בהם כך שסך ניקוד היעדים יהיה מקסימלי, תחת מגבלות מסוימות שלא ניתן לחצות (מגבלות קשיחות).

האלגוריתם הנבחר עבור הפרויקט הוא אלגוריתם ה-DPSO. הסיבה לבחירה בו היא שהוא מאפשר לאזן בין איכות הפתרון לבין זמן חישוב הפתרון באופן הכי טוב מבין האלגוריתמים המוצעים ומאפשר גמישות וכיוונון של הפרמטרים על מנת להגיע לאיזון זה בהתאם ליחס איכות לעומת זמן חישוב.

פתרון הבעיה האלגוריתמית מיושם לפי מספר שלבים מוגדרים של האלגוריתם הנבחר (DPSO):



להלן הסבר על כל שלב שמופיע בתרשים:

שלב 1 – בניית פתרון התחלתי איכותי

יש לייצר פתרונות התחלתיים שישמשו נקודת פתיחה טובה לתהליך האופטימיזציה. מטרת שלב זה היא ליצור מסלול ישים - כלומר, כזה שעומד במגבלות הנתונות - תוך ניסיון לנצל ערכים גבוהים ולבנות מבנה ראשוני בעל פוטנציאל לשיפור. פתרון התחלתי איכותי מאפשר לאלגוריתם להימנע מאזורים גרועים במרחב החיפוש ולכוון את תהליך האופטימיזציה לעבר מסלולים מבטיחים כבר מהצעדים הראשונים.

שלב 2 – הפקת פתרונות חדשים על בסיס ניסיון עבר

בשלב זה נוצרים מסלולים חדשים באמצעות שילוב בין מסלול קיים לבין מידע שנצבר מפתרונות מוצלחים בעבר. הרעיון הוא לנצל דפוסים טובים שכבר זוהו - כמו צמתים שהופיעו במסלולים

איכותיים או סדרי ביקור שהוכיחו עצמם - ולשלב אותם בתוך מסלולים מועמדים חדשים. תהליך זה מאפשר לאלגוריתם להרחיב את אזורי החיפוש המבטיחים ולייצר וריאציות מתקדמות המבוססות על ידע שנצבר לאורך הדרך. לצורך כך, עבור כל חלקיק נבנית רשימה זמנית הנקראת Ω (אומגה). עבור החלקיק הנוכחי, עוברים על כל צומת במסלול שלו. מגרילים מספר אקראי Λ בין 0 ל-1, ובודקים אותו מול שלושה פרמטרים: w , c_1 ו- c_2 . הפרמטר w מייצג את רכיב האינרציה, והוא קובע את הסיכוי לשמור צמתים מהמסלול הנוכחי; הפרמטר c_1 הוא הרכיב הקוגניטיבי, והוא שולט בסיכוי לבחור צמתים שעבדו טוב עבור החלקיק בעבר (p_{ibest}); הפרמטר c_2 הוא רכיב הלמידה החברתית, והוא קובע את המשיכה למסלול הטוב ביותר הידוע בכלל הקבוצה (g_{best}). ובכך נבנית רשימת הצמתים Ω , שהיא תערובת הסתברותית של חלקי מסלולים מוצלחים, שלאחר ניקוי כפילויות הופכת לבסיס שממנו נבנה מסלול חדש. מסלול זה נבדק תחילה לשימויות בהתאם למגבלת הזמן; ואם הוא חורג, האלגוריתם מוחק ממנו צמתים עד שהפתרון הופך בר-ביצוע.

שלב 3 – שיפור מקומי של מסלולים מועמדים

לאחר יצירת מסלולים חדשים, מתחיל שלב שיפור מקומי (local search) הנקרא RVNS (Reduced Variable Neighborhood Search), שמטרתו לחדד את איכות הפתרונות ללא שינוי מבני עמוק. ה-local search מוגדר באמצעות שתי פעולות בסיסיות: insert, המנסה להוסיף למסלול צומת חדש שאינו מופיע בו כל עוד מגבלת המרחק מאפשרת, ו-exchange, המחליפה צומת קיים בצומת אחר בעל תרומה אפשרית גבוהה יותר. מטרת שני המנגנונים היא לאתר שינויים קטנים שיכולים לשפר משמעותית את הניקוד הכולל של המסלולים.

שלב 4 – ייצוב ושיפור מבני המסלול

בשלב זה התמקדות היא באופטימיזציה של מבנה המסלול עצמו באמצעות פעולת opt-2. המטרה היא לצמצם את אורך המסלול ולשפר את הרציפות והיעילות הגאומטרית שלו. התאמות אלה עשויות לאפשר הקטנת עלות כוללת תוך שמירה על הצמתים החשובים, ולעיתים אף לפתוח אפשרות להוסיף נקודות ביקור נוספות בעתיד. השלב מביא לכך שהמסלול הופך קצר, יציב ויעיל יותר מבחינה מרחבית. פעולת ה-opt-2 עובדת באמצעות החלפת זוג קשתות כך שמסלול חלקי מתהפך, במטרה לקצר את הדרך ולהפחית את עלות הנסיעה הכוללת. אם פעולת opt-2 הצליחה לצמצם את אורך המסלול, האלגוריתם מנסה שוב להוסיף צמתים רווחיים באמצעות insert נוסף, כדי לנצל את "הזמן שהתפנה". המסלול המשופר מתקבל כ-position החדש של החלקיק, ואם הוא טוב יותר מה- p_{ibest} - הוא מחליף אותו; ואם הוא טוב יותר מכל מסלול אחר באוכלוסייה - הוא הופך ל- g_{best} .

שלב 5 – בדיקת ישימות והתאמה למגבלות

לאחר ביצוע השיפורים, נדרש לוודא שהמסלול החדש עדיין עומד בכל אילוצי הבעיה המתמטית. אם השיפור המקומי או המבני הוביל לחריגה ממגבלת האורך, יש לבצע התאמות המבטיחות חזרה לפתרון ישים. תהליך זה שומר על חוקיות הפתרון לאורך כל שלבי החיפוש ומבטיח שהאלגוריתם אינו סוטה מהגדרות הבעיה.

שלב 6 – בחירת הפתרונות המתאימים והתקדמות לדור הבא

בשלב זה בוחנים את הפתרונות שהתקבלו ומחליטים אילו מהם ימשיכו להשפיע על החיפוש בהמשך. המטרה היא למצוא איזון בין שמירה על פתרונות איכותיים שכבר הובילו לשיפור משמעותי לבין עידוד של חיפוש חדש באזורים לא נחקרים. כך האלגוריתם שומר על מגוון פתרונות ולא נתקע במינימום מקומי.

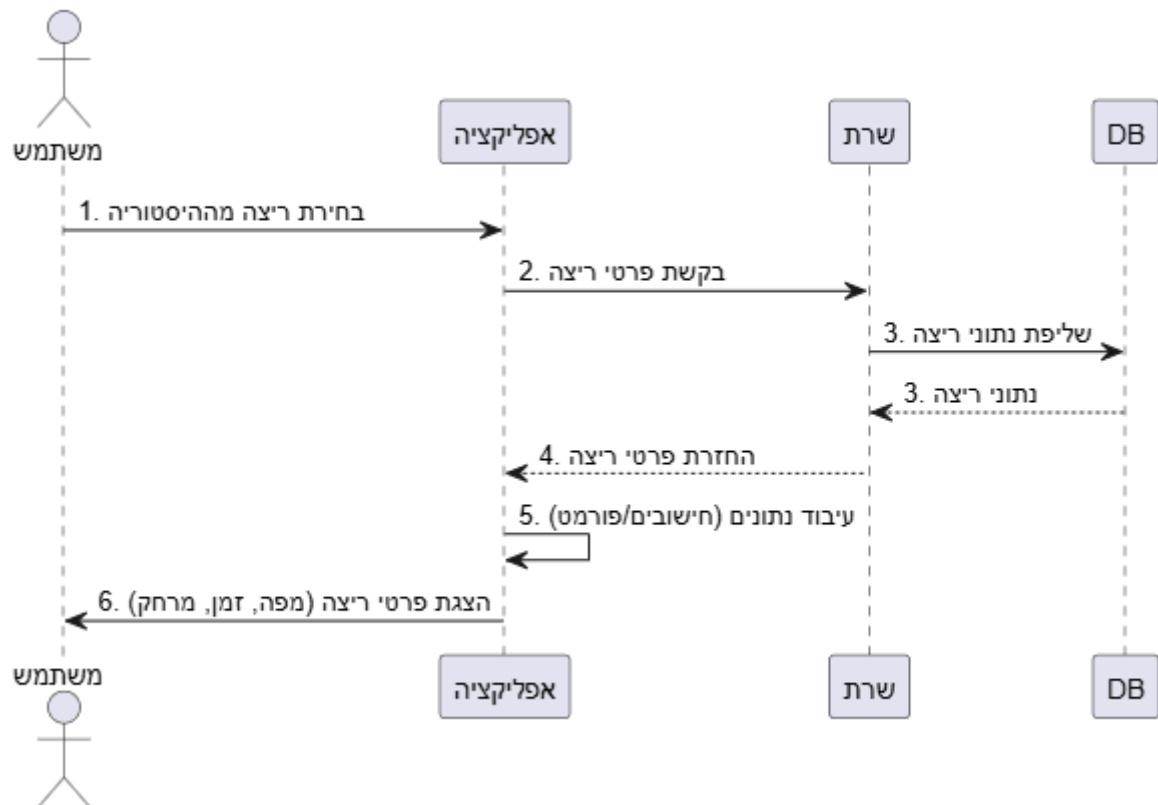
שלב 7 – עצירת החיפוש והחזרת הפתרון הטוב ביותר

כאשר מגיעים לתנאי העצירה - מספר מוגדר מראש של איטרציות ללא שיפור או מגבלת זמן - האלגוריתם עוצר ומחזיר את המסלול הטוב ביותר שנמצא לאורך כל התהליך. מסלול זה מייצג את השילוב בין ניקוד גבוה, עמידה באילוצים ויעילות מרחבית, ומשמש כפתרון הסופי לבעיית ה-Orienteeing.

5. הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות

כפי שצוין בסעיף 3, למשתמש יהיו מספר פעולות מרכזיות אשר יוכל לבצע באפליקציה.

להלן **תרשים רצף Sequence Diagram** המתאר את אחד מהתהליכים המרכזיים שצוינו בסעיף 3 - תהליך החלת תיעוד הריצה (Show Run Details) בהיבט התכנותי:

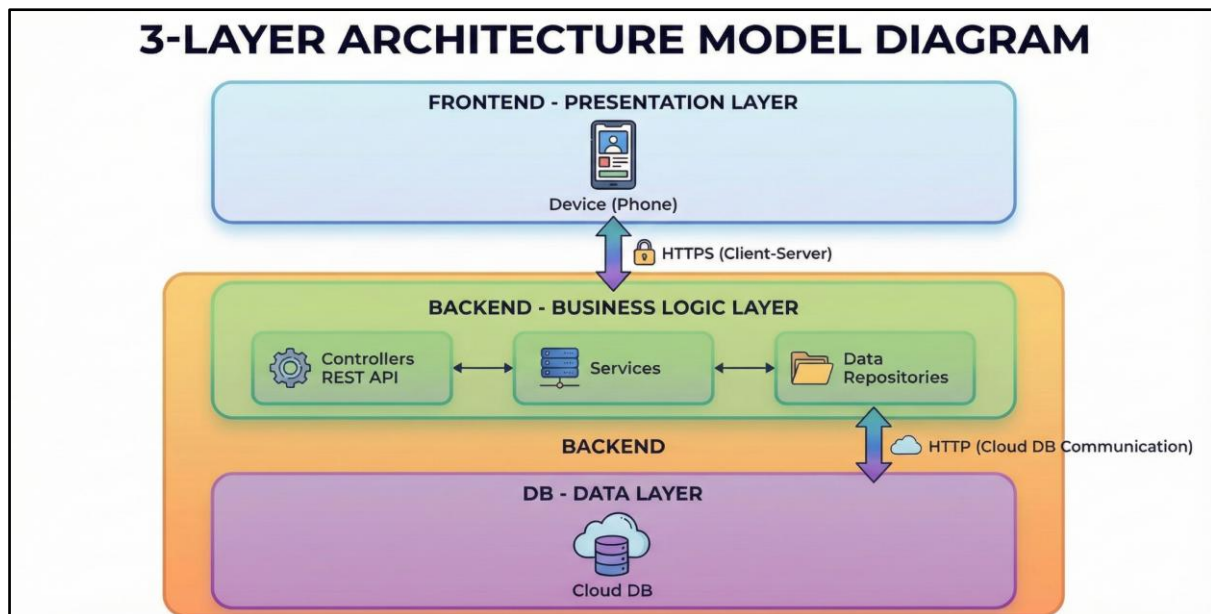


בתרשים רצף זה מתואר התהליך שבו המשתמש מציג פרטי ריצה שנשמרה בעבר באפליקציית מעקב הריצה. התהליך מתחיל כאשר המשתמש בוחר ריצה מסוימת מתוך היסטוריית הריצות שלו באפליקציה. פעולה זו מועברת לאפליקציה, אשר מזהה את הריצה שנבחרה באמצעות ומכינה בקשה לקבלת המידע הרלוונטי.

לאחר מכן האפליקציה שולחת בקשה לשרת לקבלת פרטי הריצה. השרת, האחראי על ניהול הנתונים ושמירתם, פונה למסד הנתונים ושולף את כל נתוני הריצה המבוקשת, כגון מסלול, זמן ריצה, מרחק ונתונים נוספים שנשמרו. מסד הנתונים מחזיר את המידע לשרת, והשרת מעביר אותו חזרה לאפליקציה בצורה מאורגנת.

בשלב האחרון האפליקציה מעבדת את הנתונים שהתקבלו מהשרת, מבצעת חישובים או התאמות נדרשות (כגון עיצוב המידע להצגה, חישוב קצב ממוצע או הצגת המסלול על מפה), ולבסוף מציגה למשתמש את פרטי הריצה בצורה גרפית וברורה. כך נשמרת הפרדה ברורה בין אחריות האפליקציה להצגת המידע לבין אחריות השרת לניהול ושליפת הנתונים.

האפליקציה תיבנה לפי **מודל שרת-לקוח (Client-Server Model)** המחלק בין צד השרת לצד הלקוח, משפר אבטחה, מאפשר תחזוקה קלה יותר, ותומך בהרחבה עתידית של המערכת ללא תלות ישירה בין הצדדים. המודל ימומש על ידי **ארכיטקטורת 3 השכבות** על מנת לאפשר הפרדה, סדר וחוסר תלות בין המשתמש, השרת ומסד הנתונים:



התרשים מציג את ארכיטקטורת 3 השכבות שבה המערכת מחולקת לשלושה אזורים עיקריים:

שכבת תצוגה Presentation Layer אשר נמצאת בצד ה-**Frontend** (צד הלקוח), ובמקרה זה מכשיר קצה כגון טלפון נייד. שכבה זו אחראית על הצגת המידע למשתמש, קבלת קליטים ויזואליים וביצוע פעולות משתמש. היא אינה מבצעת חישובים עסקיים ואינה ניגשת למסד הנתונים באופן ישיר, אלא פועלת מול השרת בלבד.

שכבת הלוגיה העסקית (Business Logic) שייכת לאזור ה-**Backend** (צד השרת). בתוך שכבת הלוגיקה העסקית נמצאים רכיבי ה-**Controllers (REST API)**, המשמשים כנקודת הכניסה לבקשות המגיעות מה-**Frontend**. ה-**Controllers** מקבלים בקשות HTTP, מבצעים בדיקות בסיסיות ומעבירים את הטיפול לרכיב ה-**Services**. רכיב ה-**Services** מכיל את הלוגיקה העסקית עצמה – חוקים, תהליכים וחישובים המגדירים את אופן פעולת המערכת. רכיב ה-**Data Repositories** אחראי על ניהול הגישה לנתונים, ומתווך בין הלוגיקה העסקית לבין מסד הנתונים, תוך הסתרת פרטי המימוש של שכבת הנתונים מהשכבות הגבוהות יותר.

שכבת הנתונים (Data Layer), אשר שייכת גם היא לצד ה-**Backend**, אחראית על אחסון הנתונים באופן קבוע. התקשורת בין רכיב ה-**Repositories** לבין מסד הנתונים מתבצעת באמצעות תקשורת רשת (כגון HTTP או פרוטוקול ייעודי למסדי נתונים בענן), ומאפשרת שליפה, עדכון ושמירה של מידע בצורה מבוקרת ומאובטחת.

6. הליכים עיקריים בתחום למידת מכונה - לא רלוונטי

הצעה זו לא עוסקת בלמידת במכונה ולכן סעיף זה לא רלוונטי.

7. הליכים עיקריים במע' הפעלה/רשתות/תקשורת/אבטחה - לא רלוונטי

הצעה זו לא עוסקת במע' הפעלה / רשתות / תקשורת / אבטחת מידע, ולכן סעיף זה לא רלוונטי. הערה חשובה: ההתייחסות למודל Client/Server מופיעה בהרחבה בסעיף 5 של ההצעה.

8. תיאור פרוטוקולי תקשורת

- **HTTP/HTTPS** (על גבי שכבת התעבורה TCP/IP) - פרוטוקול תקשורת לאבטחת מידע ברשתות מחשבים באינטרנט. הוא אינו פרוטוקול תקשורת כשלעצמו, אלא משתמש בפרוטוקול HTTP על שכבת TLS ובכך מקנה יכולות אבטחה של ההצפנה לתקשורת HTTP רגילה. הסיבה לשימוש בפרוטוקול על גבי שכבת TCP/IP היא בגלל היותו אמין מבחינת שמירה על איכות המידע ושלמותו.
- בנוסף לכך המערכת תכלול בתוכה שימוש ב-**REST API** (עוד לא ידוע אילו נקודות קצה) - ממשק תקשורת בין מערכות תוכנה דרך האינטרנט, המבוסס על פרוטוקול HTTP. הוא מאפשר לאפליקציות לשלוח ולקבל נתונים באמצעות פעולות סטנדרטיות כמו GET, POST, PUT ו-DELETE, בצורה פשוטה ועקבית.
- המידע ישלח על ידי פורמט **JSON** - פורמט טקסט קל משקל לייצוג והעברת נתונים. הוא בנוי ממבנה של מפתחות וערכים, קריא לאדם וקל לעיבוד על-ידי מערכות.

9. פיתוחים עתידיים

- בניית תוכנית אימונים דינמית לפי התקדמות (Adaptive Training Plan).
- ניתוח עומס ריצה והתאוששות - מעקב אחרי נתוני האימון (כמו משך הריצה, קצב, דופק ותדירות אימונים) כדי להעריך את רמת העומס שהגוף חווה. על בסיס הנתונים האלו האפליקציה מספקת חיווי לגבי מצב ההתאוששות של הרץ וממליצה האם לנוח, לבצע אימון קל או להמשיך לאימון עצים – במטרה לשפר ביצועים ולמנוע פציעות.
- מערכת התראות חכמה מותאמת אישית - מנגנון חכם שמבין את ההקשר של המשתמש ויודע לשלוח התראות מותאמות בהתאם להרגלים, שעה ביום, מזג אוויר והרגלי שימוש באפליקציה.

10. תיאור טכנולוגיית הנדסה

- הפרויקט יבנה באמצעות **שפת התכנות Java 21** - שפת עילית מבוססת OOP - משום שהיא שפה מונחית עצמים (OOP), המאפשרת בנייה מסודרת ומודולרית של האפליקציה. עקרונות אלו מקלים על ניהול ישויות, משפרים את קריאות הקוד ומאפשרים תחזוקה והרחבה עתידית של המערכת. בנוסף Java, היא השפה המרכזית לפיתוח אנדרואיד, תומכת היטב בעבודה עם GPS וחיישנים, ומספקת יציבות, ביצועים טובים ואקוסיסטם רחב של ספריות וכלים.
- מסגרת העבודה **Spring Boot Framework** (גרסה 3.5.8) תשמש עבור פיתוח צד השרת. הסיבה לשימוש ב-Spring Boot היא מאחר והוא מספק תשתית סדורה וברורה לבניית יישומי צד שרת מודרניים בשפת Java, תוך שמירה על ארכיטקטורה נקייה והפרדה נכונה בין שכבות המערכת. מסגרת העבודה מעודדת כתיבת קוד קריא, תחזוקתי וניתן להרחבה, ומאפשרת פיתוח עקבי ואחיד גם בפרויקטים גדולים ומורכבים. בנוסף, היא תומכת בבניית מערכות יציבות וסקיילביליות, מאפשרת התאמה קלה לשינויים עתידיים בדרישות, ומבוססת על עקרונות תכנון מוכחים הנהוגים בתעשייה, דבר ההופך אותו לבחירה מתאימה לפיתוח מערכות Backend בקנה מידה משתנה.
- **Visual Studio Code** הוא סביבת העבודה (IDE) שתשמש עבור פיתוח ה-Backend עם מסגרת העבודה Spring Boot. הסיבה לבחירה היא בגלל שזו סביבת פיתוח קלה, מהירה וגמישה, המאפשרת עבודה נוחה ויעילה גם בפרויקטים בצד השרת. סביבת העבודה תומכת בכתיבת קוד מסודרת וקריאה, מעודדת זרימת עבודה חלקה ומקלה על תחזוקת הפרויקט לאורך זמן. בנוסף, VS Code מאפשר התאמה אישית גבוהה לסגנון העבודה של המפתח, תומך בפיתוח חוצה פלטפורמות ומשתלב היטב בתהליכי פיתוח מודרניים, דבר ההופך אותו לבחירה מתאימה לפיתוח יישומי Backend בסביבות משתנות.
- סביבת העבודה **Android Studio** תשמש לצורך בניית צד הלקוח, מאחר שזו סביבת הפיתוח הרשמית והייעודית לאנדרואיד, המאפשרת בניית ממשק משתמש המותאם באופן מיטבי לפלטפורמה ולמכשירי הקצה. היא מספקת תהליך עבודה סדור ואחיד, תומכת בארכיטקטורה מסודרת של צד הלקוח ומעודדת כתיבת קוד קריא, תחזוקתי וניתן להרחבה. בנוסף, השימוש ב-Android Studio מאפשר התאמה טובה למחזור חיי האפליקציה, ניהול נכון של אינטראקציות עם המשתמש והבטחת חוויית שימוש עקבית, יציבה ובעלת ביצועים טובים, בהתאם לסטנדרטים המקובלים בפיתוח אפליקציות אנדרואיד.

11. מסד נתונים

המערכת תשתמש במסד נתונים בענן מסוג NoSQL.

מסד נתונים בענן הוא שירות נתונים המנוהל על-ידי ספק ענן, המאפשר אחסון, ניהול, עיבוד וגישה לנתונים דרך האינטרנט, ללא צורך בהתקנה או תחזוקה של תשתית מקומית. הסיבה לשימוש במסד נתונים בענן היא משום שהוא מספק סנכרון נתונים בזמן אמת בין מכשירים, מאפשר עבודה גם ללא שרת ייעודי - מה שחוסך תחזוק שרת, התקנת עדכונים, ניהול אבטחה וגיבויים. בנוסף, מסד בענן מעניק זמינות גבוהה ומתמודד היטב עם עומס בקשות מהמשתמשים.

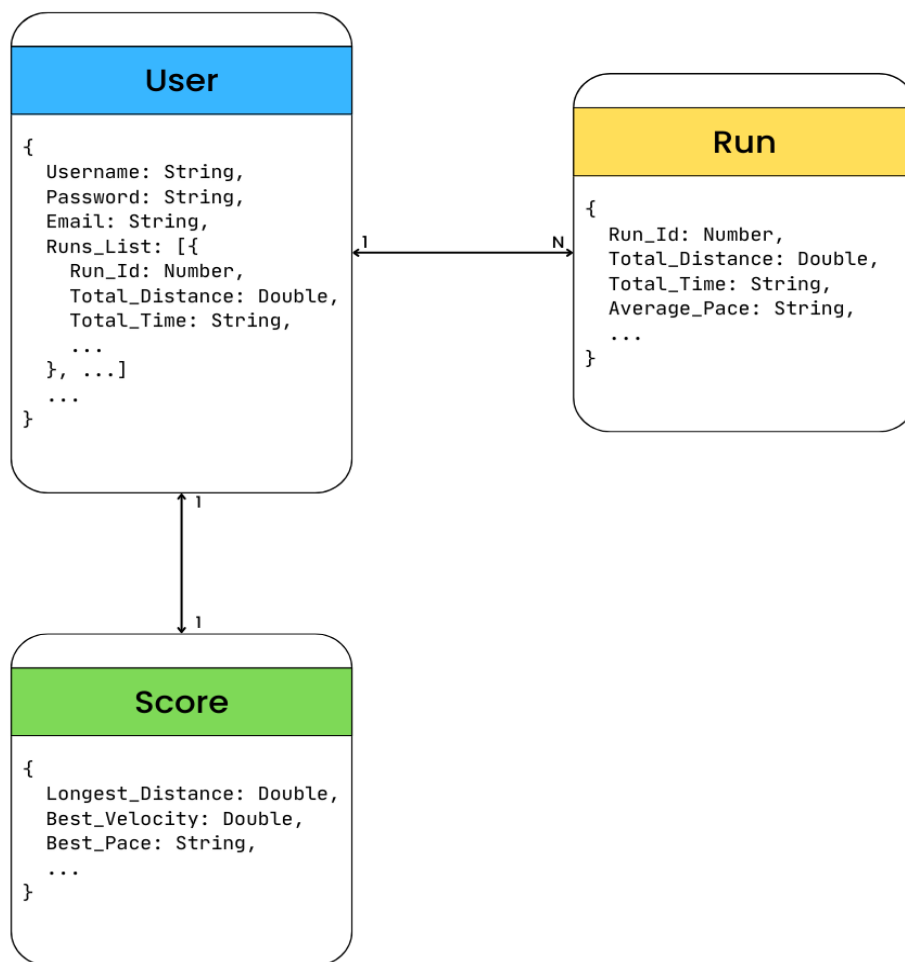
מסד נתונים NoSQL הוא סוג של מסד נתונים שלא משתמש במבנה הטבלאי הקלאסי של SQL, אלא שומר ומנהל נתונים בצורה גמישה יותר, המתאימה במיוחד לכמויות גדולות של מידע, נתונים לא-מובנים או משתנים במהירות. והסיבה לשימוש במסד נתונים מסוג NoSQL היא בגלל שמסדי נתונים מסוג זה מציעים ביצועים מהירים מאוד בקריאה וכתיבה למסד הנתונים והם בעלי תמיכה בעבודה במצב Offline, כך שמסד הנתונים ממשיך לתפקד גם בלי חיבור לרשת. כל זה הופך את הפיתוח לפשוט, יציב ויעיל יותר. דוגמה למסדי נתונים בענן ו-NoSQL הקיימים כיום - Firebase ו-MongoDB. MongoDB. המערכת תעשה שימוש ב-MongoDB מאחר והוא מאפשר שליפה והכנסת נתונים בפורמט JSON שהוא מסודר וקל להבנה.

במסד הנתונים יאוחסנו הנתונים הבאים:

- 1. משתמשים** - עבור המשתמש ישמרו שם המשתמש שלו, אימייל, סיסמה, מגדר, גיל, גובה, משקל, תאריך יצירת החשבון וכו'.
- 2. ריצות** - מסלול הריצה, תאריך, נתונים וסטטיסטיקות של הריצה - מרחק מצטבר, קצב כולל מיטבי/ממוצע/פחות, קצב לכל קילומטר, זמן ריצה כולל ועבור כל קילומטר, מהירות, וכו'.
- 3. שיאים** - שיא מרחק, שיא מהירות, שיא קצב וכו'.

יש לציין כי רשימה זו של נתונים אינה סופית ותיתכן הוספה/הורדה של נתונים במידת הצורך.

להלן **תרשים ERD** התחלתית של הישויות והקשרים שלהן:



עבור כל משתמש יש מספר ריצות ולכן הקשר ביניהם הוא N:1. הישות שיא (Score) מייצגת את ההישג האישי הכי טוב עד כה של כל משתמש כך שלכל משתמש יש רק שיא אחד המכיל בתוכו את ההישגים הכי טובים שלו עד כה.

12. פרטים פורמליים

לוחות זמנים

לסיים עד תאריך	שלבי עבודה
1.12.2025	בחירת פרויקט, חקירה ולמידה לעומק של נושאי הפרויקט
11.12.2025	כתיבה והגשת הצעת הפרויקט לאישור משרד החינוך
13.1.2026	מימוש הקוד של האלגוריתם המרכזי, ביצוע בדיקות ושיפורים
3.2.2026	בניית צד שרת
17.2.2026	בניית מסד הנתונים ושילובו
3.3.2026	בניית צד לקוח
24.3.2026	כתיבת ספר הפרויקט
7.4.2026	הגשת הפרויקט כולו (ספר + קוד) להגנה וקבלת ציון מגן

מנחה הפרויקט: _____



חתימת הסטודנט: _____

חתימת רכז המגמה: _____